

Тема 7 – Алгоритми за сортиране

При алгоритмите за сортиране, базирани на сравнение, елементите от масив се сравняват един с друг, за да се определи кой от двата елемента трябва да се постави първи в крайния сортиран списък. Всички базирани на сравнение алгоритми за сортиране имат долна граница на сложност от **$n \log n$** .

Има алгоритми за сортиране, които използват специална информация за ключове и операции, различни от сравнение, за да определят сортирания ред на елементите. Следователно долната граница на $n \log n$ не се прилага за тези алгоритми за сортиране.

Тема 7 – Алгоритми за сортиране

Алгоритъмът за сортиране е **In-place**, ако алгоритъмът не използва допълнително пространство за манипулиране на входа, но може да изисква малко, макар и непостоянно допълнително пространство за своята работа. Или можем да кажем, че алгоритъмът за сортиране сортира на място, ако само постоянен брой елементи от входния масив се съхраняват извън масива.

Алгоритъмът за сортиране е **стабилен**, ако не променя реда на елементите с една и съща стойност.

Онлайн/офлайн: Алгоритъмът, който приема нов елемент, докато процесът на сортиране продължава, се нарича алгоритъм за **онлайн** сортиране.

Тема 7 – Алгоритми за сортиране чрез размяна (Exchange sorts)

1. Метод на мехурчето (Bubble sort),
2. Exchange sort
3. Cocktail shaker sort,
4. Odd–even sort,
5. Quicksort,
6. Gnome sort.

[15 Sorting Algorithms in 6 Minutes](#)

Тема 7 – Алгоритми за сортиране чрез размяна (Exchange sorts)

Метод на мехурчето (Bubble sort)

Сортирането с **метод на мехурчето**, понякога наричано потъващо сортиране, е прост алгоритъм за сортиране, който многократно преминава през входния списък (масив) елемент по елемент, сравнявайки текущия елемент с този след него, разменяйки техните стойности, ако е необходимо. Тези преминавания през списъка се повтарят, докато не трябва да се извършват повече размени, тоест списъкът е станал напълно сортиран. Алгоритъмът е кръстен на начина, по който по-големите елементи като «мехурчета изплуват» до върха на списъка.

Този прост алгоритъм се представя слабо в реална употреба и се използва предимно като образователен инструмент, защото той има средна сложност $O(n^2)$, където n е броят на елементите, които се сортират. Повечето практични алгоритми за сортиране имат значително по-добра сложност, често $O(n \log n)$. Дори други $O(n^2)$ алгоритми за сортиране, като сортиране чрез вмъкване, обикновено работят по-бързо от него и не са по-сложни.

[Bubble Sort](#)

Тема 7 – Алгоритми за сортиране чрез размяна (Exchange sorts)

1. Exchange sort

Exchange sort сравнява първия елемент с всеки следващ елемент от масива, като прави необходимите размени. Когато първото преминаване през масива приключи, сортирането взема втория елемент и го сравнява с всеки следващ елемент от елементите на масива, които не са подредени все още.

Единствената разлика е в начина, по който се сравняват елементите. Методът на мехурчето преминава през списъка и разменя при нужда елементи, докато Exchange sort сравнява един елемент с всички останали елементи.

[Exchange sort](#)

Тема 7 – Алгоритми за сортиране чрез размяна (Exchange sorts)

Exchange sort работи, като сравнява първия елемент с всички следващи елементи, разменяйки, където е необходимо, като по този начин гарантира, че след първото преминаване през масива, първият елемент ще е на правилното място за крайния ред на сортиране; след това продължава да прави същото за втория елемент и т.н.

На този метод му липсва предимството, което има при метода на мехурчето да открива с едно преминаване през масива, ако той вече е сортиран, но все пак **Exchange sort** може да бъде по-бърз от метода на мехурчето с постоянен коефициент (едно преминаване по-малко върху данните за сортиране; наполовина по-малко общи сравнения) в най-лошия случай. Като всяко просто $O(n^2)$ сортиране, то може да бъде сравнително бързо при много малки набори от данни, въпреки че като цяло сортирането чрез вмъкване ще бъде по-бързо.

Тема 7 – Алгоритми за сортиране чрез размяна (Exchange sorts)

```
#include<stdio.h>

int main(void)
{
    int array[5], i, j, temp,n;
    printf("Enter the number of elements\n: ");
    scanf("%d",&n);
    printf("Enter %d numbers : ",n);
    for (i = 0; i < n; i++)
    {
        scanf("%d",&array[i]);
    }

    for(i = 0; i < (n -1); i++)
    {
        for (j=(i + 1); j < n; j++)
        {
            if (array[i] > array[j])
            {
                temp = array[i];
                array[i] = array[j];
                array[j] = temp;
            }
        }
    }

    printf("Sorted array is : ");
    for (i = 0; i < n; i++)
    {
        printf(" %d ",array[i]);
    }
    return 0;
}
```

Тема 7 – Алгоритми за сортиране чрез размяна (Cocktail shaker sort)

Cocktail shaker sort е друга разновидност на сортирането метода на мехурчето. При техниката на сортиране с метода на мехурчето винаги се търси отляво надясно и се намира най-големия елемент и се поставя в края на списъка, като във втората фаза намира втория по големина елемент и го позиционира на предпоследната позиция.

Cocktail shaker sort преминава алтернативно в двете посоки. Тук в най-лошият случай времева сложност е $O(n^2)$.

[Cocktail Shaker Sort](#)

Тема 7 – Алгоритми за сортиране чрез размяна (Cocktail shaker sort)

```
#include<iostream>

using namespace std;

// A function to swap values using call by reference.
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

// A function implementing shaker sort.

void ShakerSort(int a[], int n)

{ int i, j, k;

for(i = 0; i < n;) {

 // First phase for ascending highest value to the highest unsorted index.

 for(j = i+1; j < n; j++) {

 if(a[j] < a[j-1])

 swap(&a[j], &a[j-1]); }

 // Decrementing highest index.

 n--;

 // Second phase for descending lowest value to the lowest unsorted index.

 for(k = n-1; k > i; k--) {

 if(a[k] < a[k-1])

 swap(&a[k], &a[k-1]); }

 // Incrementing lowest index.

 i++; } }

```

int main()
{
    int n, i;
    cout<<"\nEnter the number of data element to be sorted: ";
    cin>>n;
    int arr[n];
    for(i = 0; i < n; i++)
    {
        cout<<"Enter element "<<i+1<<": ";
        cin>>arr[i];
    }
    ShakerSort(arr, n);
    // Printing the sorted data.
    cout<<"\nSorted Data ";
    for (i = 0; i < n; i++)
        cout<<"->"<<arr[i];
    return 0;
}

```

Cocktail shaker sort

Тема 7 – Алгоритми за сортиране чрез размяна (Cocktail shaker sort)

Първото преминаване вдясно ще премести най-големия елемент на правилното му място в края, а следващото преминаване вляво ще премести най-малкия елемент на правилното му място в началото. Второто пълно преминаване ще премести втория по големина и втория най-малък елемент на правилните им места и т.н.

След i на брой итерации, първите i и последните i елементи в списъка са на правилните си позиции и не е необходимо да се проверяват. Това води до съкращаване на частта от списъка, която се сортира всеки път, като броят на операциите може да бъде намален наполовина.

Тема 7 – Алгоритми за сортиране чрез размяна (Odd–even sort)

Odd–even sort (Четно-нечетно сортиране, odd–even transposition sort, brick sort, parity sort) е сравнително прост алгоритъм за сортиране, разработен първоначално за използване на паралелни процесори с локални взаимовръзки .

Това е сортиране със сравнение, близко по алгоритъм до сортирането с метода на мехурчето. **Четно-нечетно сортиране** функционира, като сравнява всички нечетни/четни индексирани двойки съседни елементи в списъка и, ако една двойка е в грешен ред (първият елемент е по-голям от втория), елементите се сменят. Следващата стъпка повтаря това за четни/нечетни индексирани двойки (от съседни елементи). След това се редува между нечетни/четни и четни/нечетни стъпки, докато списъкът бъде сортиран.

Този алгоритъм има $O(n^2)$ сложност.

(Odd-even sort)

```
#include<iostream>

using namespace std;

void brickSort(int arr[], int n) {
    bool flag = false;
    while(!flag)
    {
        flag = true;
        for(int i = 1; i<n-1; i= i+2)
        {
            if(arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                flag = false;
            }
        }
        for(int i = 0; i<n-1; i= i+2)
        {
            if(arr[i] > arr[i+1]) {
                swap(arr[i], arr[i+1]);
                flag = false;
            }
        }
    }
}
```

Odd-even sort

```
int main()
{
    int data[] = {54, 74, 98, 154, 98, 32, 20, 13, 35, 40};
    int n = sizeof(data)/sizeof(data[0]);
    cout << "Sorted Sequence ";
    brickSort(data, n);
    for(int i = 0; i < n; i++)
    {
        cout << data[i] << " ";
    }

    return 0;
}
```

Odd-even sort (Brick Sort)

Тема 7 – Алгоритми за сортиране чрез размяна (Quicksort)

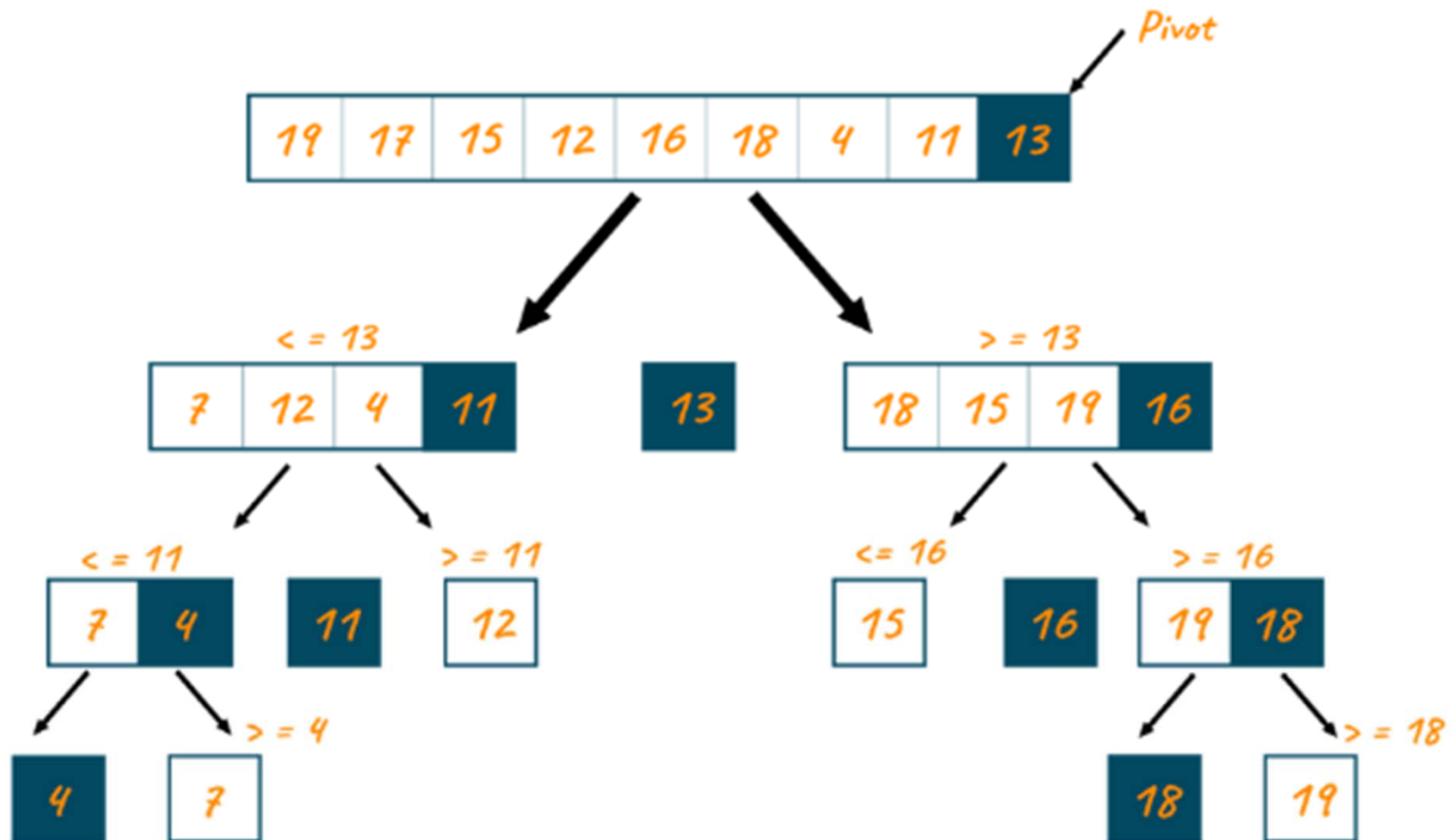
Quicksort (Бързо сортиране) е алгоритъм "разделяй и владей". Той работи, като избира елемент „пивот“ (pivot) от масива и разделя другите елементи на два подмасива, според това дали са по-малки или по-големи от пивота. Поради тази причина понякога се нарича сортиране чрез разделяне.

След това подмасивите се сортират рекурсивно. Това може да се направи на място, като са необходими малки допълнителни количества памет за извършване на сортирането.

Математическият анализ на бързото сортиране показва, че средно алгоритъмът отнема $O(n \log n)$ сравнения, за да сортира n елемента. В най-лошия случай той прави $O(n^2)$ сравнения.

Тема 7 – Алгоритми за сортиране чрез размяна (Quicksort)

Quick Sort Algorithm



Quicksort

```
#include <iostream>
using namespace std;
void swap(int arr[], int pos1, int pos2) {
    int temp;
    temp = arr[pos1];
    arr[pos1] = arr[pos2];
    arr[pos2] = temp;
}

int partition(int arr[], int low, int high, int pivot) {
    int i = low;
    int j = high;
    while( i <= high) {
        if(arr[i] > pivot) {
            i++;
        }
        else {
            swap(arr,i,j);
            i++;
            j--;
        }
    }
    return j-1;
}
```

Quicksort

```
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pivot = arr[high];
        int pos = partition(arr, low, high, pivot);

        quickSort(arr, low, pos-1);
        quickSort(arr, pos+1, high);
    }
}
```

Quicksort

```
int main()
{
    int n;
    cout << " enter the size of array";
    cin>>n;
    int arr[n];
    for( int i = 0 ; i < n; i++)
    {
        cin>> arr[i];
    }
    quickSort(arr, 0 , n-1);
    cout<<"The sorted array is: ";
    for( int i = 0 ; i < n; i++)
    {
        cout<< arr[i]<<"\t";
    }
}
```

[Quicksort](#)

Тема 7 – Алгоритми за сортиране чрез размяна (Gnome sort)

Gnome sort е вариант на алгоритъм за сортиране чрез вмъкване, който не използва вложени цикли. Първоначално **Gnome sort** работи, като изгражда сортиран списък елемент по един елемент, поставяйки всеки елемент на правилното място с поредица от размени. Средното време на работа е $O(n^2)$, но клони към $O(n)$, ако списъкът първоначално е почти сортиран.

[Gnome sort](#)

Тема 7 – Алгоритми за сортиране чрез размяна (Gnome sort)

Gnome Sort се основава на техниката, използвана от стандартния холандски градински гном (на английски: *tuinkabouter*).

Ето как един градински гном сортира редица саксии.

По принцип той гледа саксията с цветя до него и предишната; ако са в правилния ред, той пристъпва една саксия напред, в противен случай ги разменя и се връща една саксия назад.

Гранични условия: ако няма предишена саксия, той пристъпва напред; ако няма саксия до него, той е готов.

Gnome sort

```
#include <iostream>using
namespace std;
// A function to sort the algorithm using gnome sort
void gnomeSort(int arr[], int n)
{
    int index = 0;
    while (index < n)
    {
        if (index == 0)
            index++;
        if (arr[index] >= arr[index - 1])
            index++;
        else
        {
            swap(arr[index], arr[index - 1]);
            index--;
        }
    }
    return;
}
```

Gnome sort

```
// A utility function to print an array of size n
void printArray(int arr[], int n)
{
    cout << "Sorted sequence after Gnome sort: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << "\n";
}

// Driver program to test above functions.
int main()
{
    int arr[] = { 34, 2, 10, -9 };
    int n = sizeof(arr) / sizeof(arr[0]);
    gnomeSort(arr, n);
    printArray(arr, n);
    return (0);
}
```